

Introduction

👋 Welcome to the **Hardened Images Launch** lab! During this lab, you will learn to do the following:

- Pull & Run Hardened Images
- Do Multi-stage build with Hardened Images
- Use Compose with Hardened Images

Throughout this lab you will use a repository containing a Restaurant reservation (**dinner**) application consisting of a basic Python app talking to a PostgreSQL database to demonstrate the use of Hardened Images.

First things to get started, please provide following registry URLs:

What is your Hardened Images Container Registry prefix?

Update

What is your GitHub Container Registry (ghcr.io) remote repository URL?

Update

What is your DockerHub remote repository URL ?

Update

If you run this lab locally, and not within the labspace environment, you will also need to clone the project locally:

```
git clone https://github.com/mathieu-benoit/dhi-launch
cd dhi-launch/project
```

Pull & Run

Pull the Python image

Pull the hardened Python image from your private registry:

```
podman pull dhi.io/python:3.14-debian13
```

Run the Python image

Run locally this hardened Python image:

```
podman run --rm dhi.io/python:3.14-debian13 python -c 'print("Hello, Hardened Image!")'
```

Test the "No shell" variant

Try to run a shell with this hardened Python image:

```
podman run --rm -it dhi.io/python:3.14-debian13 sh
```

You get this error message:

```
OCI runtime create failed: runc create failed: unable to start container process: error during container init: exec: "sh": executable file not found in $PATH
```

You cannot run any commands because this **runtime** variant image doesn't have a shell, a package manager or any extra system packages. We will illustrate this part in the next section.

Test the access to internal website

Identify a website with your corporate issued TLS certificate:

What is a website only accessible with your corporate issued TLS certificate?

Update

```
echo "Q" | openssl s_client -connect google.com:443 -showcerts 2>/dev/null | openssl x509 -noout -issuer
```

Try to access an internal website from this hardened Python image:

```
podman run --rm dhi.io/python:3.14-debian13 python -c 'import urllib.request; print(urllib.request.urlopen("https://google.com", timeout=10).status)'
```

You should see a successful **200** response.

Multi-stage build

Update the Dockerfile to use an hardened dev base image

Change the base image in the `FROM` instruction in the [Dockerfile](#) and save.

```
FROM dhi.io/python:3.14-debian13-dev

# FROM python:3.14
# FROM dhi.io/python:3.14-debian13-dev
```

Build the updated image with the hardened `dev` variant:

```
podman build -t dinner:dev .
```

If you have Docker Scout, compare the CVEs between the initial image and the latest hardened `dev` image built locally:

```
docker scout compare \
  --ignore-unchanged \
  --to ghcr.io/mathieu-benoit/dinner:initial@sha256:a801c9e163a3834000018d5009b9323088d25461c2ceba9ed03f4cf32c3d900 \
  dinner:dev
```

See the number of CVEs, packages and size of the image just got improved:

- CVEs: -232
- Packages: -367
- Size (on disk): -1443MB

Test the "shell" variant

Try to run a shell with this hardened `dev` image built locally:

```
podman run --rm -it dinner:dev sh
```

You can run some commands because this `dev` variant image has a shell, a package manager and extra system packages.

Exit the opened shell:

```
exit
```

Update the Dockerfile to use a runtime base image

Update the [Dockerfile](#) with this content:

```
FROM dhi.io/python:3.14-debian13-dev AS builder
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
WORKDIR /app
RUN pip install psycogp2-binary --target /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt --target /app

FROM dhi.io/python:3.14-debian13 AS runtime
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
COPY --from=builder /app /app
COPY . /app
EXPOSE 5000
CMD ["python", "/app/app.py"]

podman build -t dinner:runtime .
```

If you have Docker Scout, compare the CVEs between the `dev` and the `runtime` hardened images built locally:

```
docker scout compare --ignore-unchanged --to dinner:dev dinner:runtime
```

See the number of CVEs, packages and size of the image just got improved:

- CVEs: -16
- Packages: -57
- Size (on disk): -92MB

Test the "no shell" variant

Try to run a shell with this hardened `runtime` image built locally:

```
podman run --rm -it dinner:runtime sh
```

You get this error message:

```
OCI runtime create failed: runc create failed: unable to start container process: error during container init: exec: "sh": executable file not found in $PATH
```

You cannot run any commands because this `runtime` variant image doesn't have a shell, a package manager or any extra system packages.

Note: Out of scope of this workshop, but instead, you can use the `docker debug` or `kubectl debug` commands to attach a temporary, tool-rich debug container to the running instance.

Compose

Modify the Docker Compose file to use the PostgreSQL image

Change the db image in the [compose.yaml](#) and save.

```
image: dhi.io/postgres:18-debian13
```

```
- image: postgres:18  
+ image: dhi.io/postgres:18-debian13
```

Run the application

Run the application:

```
podman compose up -d --build
```

Go to the application in the browser: <http://localhost:5001>.

Enter a reservation using the application to confirm that the application is working.

Scan the PostgreSQL image

If you have Docker Scout, compare the CVEs between the initial PostgreSQL image and the hardened PostgreSQL image:

```
docker scout compare --ignore-unchanged --to docker.io/postgres:18 dhi.io/postgres:18-debian13
```

See the number of CVEs, packages and size of the image just got improved:

- CVEs: -44
- Packages: -67
- Size (on disk): 99MB

Conclusion

You've completed the **Hardened Images Launch** lab!

✅ You now know how to:

- Pull & Run Hardened Images
- Do Multi-stage build with Hardened Images
- Use Compose with Hardened Images

🎉 Well done!

Resources

- [Multi-stage builds](#)
- [Base image hardening](#)
- [Troubleshoot hardened images](#)